

Cashier Charge

Architecture & Business Logic Specification

Restaurant Order & Billing System

Contents

1 Purpose	1
2 Operating Context	1
3 Core Design Principles	1
4 Zones (Fully Configurable)	2
4.1 Optional table numbers, decided per zone	2
4.2 Retiring a zone	2
5 The Check (Open Tab) Model	2
5.1 Check lifecycle	2
5.2 Order item state machine	2
6 Cashier Flow	3
7 The Cashier Screen	3
8 Authorization & Accountability	3
9 Data Model (Conceptual)	4
10 Reporting (Driven by the Model)	4
11 Open Questions to Confirm	4

1 Purpose

This document specifies the architecture and business logic of the **Cashier** role for a restaurant order and billing system. The cashier registers orders that servers bring from the floor, keeps each table's bill on an open check, and settles or flags it at the end of service. The design follows patterns proven in established restaurant POS systems (open-check model, item state machine, comp/void with reason codes, and audit trails).

2 Operating Context

- Servers take orders from customers on the floor, then walk to the cashier, who enters the order. *Order entry is cashier-centric.*
- Service is organized into **zones** (e.g. VIP1, VIP2, Outside). Zones are **not fixed**: the owner can add, rename, reorder, or retire them at will.
- Each server is assigned to a zone per shift, so the same zone may have a different server tomorrow.

- Two shifts per day, configurable; default $\approx$ 07:00–16:00 and 16:00–01:00.
- Some customers leave without paying, so unpaid checks must be tracked and attributed to a responsible server.

3 Core Design Principles

1. **Everything configurable is data, not code.** Zones, shifts, products, payment methods, and reason codes are records the owner manages, never hardcoded values.
2. **Reference by ID, display by name.** Checks and orders point to a zone by its ID, so renaming a zone never breaks history.
3. **Never hard-delete operational data.** Retire (deactivate) instead of delete, so past checks and reports stay intact.
4. **Every money-affecting action is attributed and logged.** Who, what, when — for accountability and theft prevention.
5. **The check is the heart of the model.** An open check accumulates order items and is closed by a payment.

4 Zones (Fully Configurable)

Instead of three fixed zones, the owner gets a **Zones** management screen. A zone is a record with:

Field	Description
zone_id	Stable internal identifier (never reused)
name	Display name (e.g. “VIP1”, “Terrace”)
display_order	Controls order on the cashier screen
table_mode	none / free / fixed (see below)
active	Shown for new orders when true

4.1 Optional table numbers, decided per zone

The **zone** is always required; the **table number** is optional and its behavior is set per zone via **table_mode**:

- **none** — the zone uses no table numbers.
- **free** — the cashier may type any table number, or leave it blank.
- **fixed** — the owner pre-defines a list of tables; the cashier picks one.

Always-present identifier. Every check receives an automatic **ticket number** regardless of table mode, so a check can always be identified even when no table number is given. The table number, when present, is extra detail on top of the ticket number.

4.2 Retiring a zone

“Removing” a zone sets **active** = **false**. It disappears from new-order screens but remains attached to historical checks, keeping reports correct. This single rule is what keeps the system from corrupting its own history.

5 The Check (Open Tab) Model

A **check** represents one customer party's running bill. It is opened on the first order and closed by payment.

5.1 Check lifecycle

1. **OPEN** — created with the first order item; zone + server + ticket number assigned.
2. **IN_PROGRESS** — additional items added across the visit.
3. **CLOSED_PAID** — settled by one or more payments.
4. **CLOSED_UNPAID** — closed without full payment (walkout / write-off); requires a reason and is flagged for the owner.
5. **VOIDED** — entire check cancelled before fulfillment (requires reason + authorization).

5.2 Order item state machine

Following standard POS practice, each order line moves through states. This is what makes corrections safe and auditable.

State	Meaning
HELD	Entered but not yet sent to kitchen/bar; freely editable.
SENT	Fired to kitchen/bar; preparation has begun.
VOID	Removed <i>before</i> preparation (mistake caught in time). No cost to customer; reason recorded.
COMP	Removed <i>after</i> preparation/delivery (goodwill or error). Item was consumed; affects inventory and food cost; reason recorded.

Void vs. Comp matters financially. A *void* is an item killed before it was made — no food used. A *comp* is an item already made or delivered, then taken off the bill — food was used but not paid for. They hit inventory and food cost differently and must be separated in reporting.

6 Cashier Flow

1. **Server identification.** Server enters a PIN (or badge). System resolves the server and their assigned zone for the current shift.
2. **Select zone.** Required. Picker is filtered to the server's zone(s).
3. **Table number.** Optional, behaves per the zone's `table_mode`.
4. **Open or reuse a check.** No matching open check → create one with a ticket number. Otherwise add to the existing check.
5. **Add items.** Tap products + quantities (+ modifiers). Each line is stamped with `server_id` and timestamp, state = **HELD**.
6. **Send.** Lines fire to kitchen/bar; state → **SENT**.
7. **Corrections.** Void (pre-prep) or comp (post-prep) with a reason code and, where required, manager authorization.
8. **Settle.** Close the check with payment(s). Any check open past shift end surfaces in the unpaid report.

7 The Cashier Screen

Four regions:

- **Server login** — PIN / badge entry; shows active server.
- **Zone & table picker** — zone required, table optional per mode; filtered to the server's zone.
- **Product grid** — categorized products with quantity and modifiers.
- **Open-check panel** — running items, per-line state, running total; actions to send, void, comp, and pay.

8 Authorization & Accountability

- Every order line carries the **responsible server** and a timestamp.
- **Voids and comps require a reason code** from an owner-managed list (e.g. kitchen error, customer dissatisfaction, spill, staff meal).
- Money-reducing actions (void, comp, unpaid close) can require **manager authorization** and are written to an audit log.
- The system can flag outliers — e.g. a server with unusually many comps or unpaid checks in a day — to deter theft.

9 Data Model (Conceptual)

Entity	Key fields
Zone	zone_id, name, display_order, table_mode, active
TableSpot	table_id, zone_id, label, active (<i>only for fixed mode</i>)
Server	server_id, name, pin_hash, active
Shift	shift_id, name, start_time, end_time
ShiftAssignment	assignment_id, server_id, zone_id, shift_id, date
Product	product_id, name, category, price, active
Modifier	modifier_id, product_id, name, price_delta
Check	check_id, ticket_number, zone_id, server_id, table_id?, status, opened_at, closed_at
OrderItem	item_id, check_id, server_id, product_id, qty, price, state, reason_code?, created_at
Payment	payment_id, check_id, method, amount, paid_at
ReasonCode	reason_id, kind (void/comp/unpaid), label, active
AuditLog	log_id, actor_id, action, target, reason_id?, timestamp

Notes: a ? marks an optional field. **Check.table_id** is null when the zone uses no table numbers or the cashier left it blank. Prices are copied onto **OrderItem** at entry time so later menu changes never alter past bills.

10 Reporting (Driven by the Model)

Because every line is attributed and time-stamped, the owner gets:

- Sales per zone, per server, per shift, per day.
- Unpaid checks, with the responsible server.
- Void and comp totals by reason code (waste and food-cost insight).
- Outlier alerts for accountability.

11 Open Questions to Confirm

- Does the cashier also take payment, or is settling a separate role?
- Should servers ever enter orders themselves (tablet), or strictly cashier-only?
- Manager authorization: required for all voids/comps, or only above a threshold?
- Target platform: web, desktop, or mobile?
- Word for the physical spot in the UI: “table” or “place”?